# Runtime Membership Contracts for SWI-PeTTa

A concise account of the runtime typing draft

6 November 2026

**Abstract**

A declared function has one checked entry. Membership checks constrain values without generating extra executions of its equations; open values retain requirements that are reconsidered when they bind. This note describes the mathematical reading, the native Prolog implementation, and the boundary between proved abstract laws and tested implementation behavior. It retains PeTTa's Atom argument and result conventions. It is a research draft, not a verified compiler or a static type system.

## 1 Why membership contracts fit this runtime

PeTTa programs manipulate symbolic terms, unify variables, enumerate alternative answers and edit spaces. A useful boundary check must therefore accept partially known data, preserve sharing, and distinguish a failed membership test from an extra answer. Treating a declaration as a predicate on values gives a direct reading of these operations. For example:

```
(: Nil (List $a))
(: Cons (-> $a (List $a) (List $a)))
(: same (-> $a $a Bool))
(= (same $x $y) True)
```

The repeated `$a` requires a common type witness. It does not prove that `same`'s implementation is universally parametric. Constructor parameters are fresh at each use, whereas all occurrences in one function signature share one assignment.

This approach draws on polymorphic logic programming [1] and the constraint-logic-programming separation between goals and constraints [2]. Those are foundations and precedents, not claims that this implementation satisfies all results of either work. Here declarations guard execution; they do not statically prove all equation bodies well typed.

SWI-Prolog supplies the mechanisms this design needs: indexed clauses for membership, unification for private type equations, and attributed variables for requirements that survive until a value binds [3]. Independent primitive checks use `freeze/2` [4]. Reflection remains separate: `get-type` may be extended by applications, but enforcement reads declarations rather than executing arbitrary reflective code.

## 2 Judgments, witnesses and explicit checking patterns

Let $v$ range over finite first-order values, with open values additionally containing program variables. Let $\Gamma$ describe a declaration view. Proper type witnesses $\tau$ are finite type expressions, and $\theta$ maps signature parameters to such witnesses. A checking pattern $p$ may also contain an explicit unrestricted check $\mathsf{any}$. These two syntactic categories must not be conflated.

Write $\Gamma \vdash v \Leftarrow p$ for checking a closed value against a closed pattern. Representative rules are:

$$\frac{v \text{ is represented as a number}}{\Gamma \vdash v \Leftarrow \mathsf{Number}} \qquad \frac{}{\Gamma \vdash v \Leftarrow \mathsf{any}} \qquad \frac{\Gamma \vdash v_i \Leftarrow p \quad (1 \le i \le n)}{\Gamma \vdash [v_1, \ldots, v_n] \Leftarrow \mathsf{ListOf}(p)}.$$

Formation is a premise: `(ListOf ListOf)` is malformed, so even an empty list must not pass it. Runtime guards for primitive representations are not casts: a declaration cannot make a symbol a Number.

Nominal facts provide membership of whole values. Constructor declarations provide membership by checking fields and matching the result family. Schematically, for a well-formed constructor scheme:

$$\frac{c : p_1, \ldots, p_n \to K(\bar{q}) \in \Gamma \qquad \Gamma \vdash v_i \Leftarrow \rho p_i \ (1 \le i \le n)}{\Gamma \vdash c(v_1, \ldots, v_n) \Leftarrow K(\rho\bar{q})}.$$

The local matching substitution $\rho$ preserves all equalities in the constructor scheme. Nominal pattern variables shared with type indices must be matched jointly. A whole-value fact gives no license to infer field invariants. Nonregular recursive families use the general checker when direct predicate generation would keep expanding.

**Where Atom belongs.** The source has distinct uses which must remain explicit:

| Position | Draft interpretation |
|---|---|
| Literal Atom input slot | Hold caller syntax; no value restriction. |
| Unanimous Atom result slots | Compile the equation body as a template. |
| Atom in a structural element/field check | Unrestricted check `any`. |
| Atom in a user-family index | Explicit index, matched exactly; no inferred covariance. |
| `_`, `%Undefined%` | Unrestricted whole slot; nested uses remain unsupported. |

Thus `(ListOf Atom)` accepts a heterogeneous list. By contrast, `(List Atom)` need not accept a value declared `(List Number)`. User-family indices are not automatically covariant. Arrow interface annotations retain their calling conventions and are matched without a general subtyping relation.

An unrestricted checking marker must never become the solution of a shared signature parameter, including inside a compound witness. Otherwise checking `(Pair $a Atom)` against a constructor with result `(Pair $x $x)` can waive the very equality it should preserve. The runtime keeps a delayed proper-witness obligation on signature variables to prevent that escape. Explicit constructor-field demands may still be unrestricted. A bare fact `(: x Atom)` supplies no distinguishing witness; an atom without usable proper nominal evidence retains its canonical Symbol fallback.

This distinction is why `(same 1 "s")` fails even though a separately written `Atom` check accepts either value. Adding an unrestricted check to the checking language does not add a universal witness to type inference.

# 3 One call, one body computation

For fully known inputs $\bar{v}$, a selected signature $d = (p_1, \ldots, p_n \to p_r)$ has the idealized staged contract

$$\mathrm{Joint}_d(\Gamma_0, \Gamma_1, \bar{v}, r) \iff \exists \theta : \mathsf{Proper}.\ \left( \bigwedge_i \Gamma_0 \vdash v_i \Leftarrow \theta p_i \right) \wedge \Gamma_1 \vdash r \Leftarrow \theta p_r.$$

Alternative declarations form a disjunction outside this judgment. They are alternative justifications for admission, not instructions to execute the body repeatedly. Entry retains the compatible alternatives; exit must not be restricted to whichever witness happened to be found first. Genuine nondeterministic equation answers retain their order and multiplicity.

The operational outline is:

$$\boxed{\text{prepare arguments}} \longrightarrow \boxed{\text{check/retain input evidence}} \longrightarrow \boxed{\text{execute worker}} \longrightarrow \boxed{\text{check result}}.$$

Evaluation modes are chosen before evaluating argument expressions. Captured partial-application arguments are already values and are not evaluated again. Direct, indirect, completed partial and native/memoized routes converge on checked entries when a contract applies. Result checks occur after the body; they must not be moved ahead of effects or exceptions. Existing output unification and head constraints are retained.

**Open values.** An unknown value produces a residual requirement rather than a fabricated value. Private type substitutions may be solved, but checking does not bind program variables to invented data. One residual formula preserves correlated arguments and alternative signatures. Its observable behavior is rejection of known contradictions and reconsideration on later bindings; it is not a complete global inhabitation decision procedure.

The displayed closed-input formula uses $\Gamma_0$ as a convenient single entry view. The native implementation is more specific: known input nodes retain evidence collected at entry, while a still-open leaf records evidence when it binds. This is not a whole-program historical snapshot. For example, if `old` and `new` both have type `A`, a call `keep(old)` may remove `old : A` and return `new`. Its retained input evidence still supports `A`, and its result is checked in the return view.

An identity result may reuse retained evidence only under the recorded generation premise. Database generations are invalidation evidence, not a semantic replacement for the input records. Their transaction behavior matters and is documented by SWI-Prolog [5].

# 4 Four small implementation units

| File | Responsibility |
|---|---|
| `types.pl` | Declaration index, membership evidence, joint frames, proper witnesses, local structural checks. |
| `type_compilation.pl` | Worker/wrapper emission, call modes, tail/suffix reuse, publication and retirement. |
| `type_membership.pl` | Indexed predicates for closed membership checks; general fallback and dependencies. |
| `type_obligations.pl` | Native attribute cells, notification, copy restoration and release. |

The first three use the existing runtime module so generated predicate names and definition-lifecycle hooks keep their interface. The attribute implementation is a Prolog module. Splitting source files introduces no dispatch layer.

A frame is `choices(Options, Stage, Identity, Stamp, Values)`. An empty option list denotes discharge. Its formulas contain conjunctions, membership demands and passive evidence. A passive input whose parameter occurs only in the result must record later bindings, but need not solve that input/result relation before a result exists.

One program variable owns an evidence cell and deduplicated watchers. A nested passive input may borrow a running caller's recorder, because that caller remains active until the nested call returns. At exit, any remaining obligation must acquire its own watchers. Native backtracking restores these updates. Copy tests must preserve shared cells, not just print the same answer.

Collection uses ordinary `findall`. Visible constraints copy with their variables; disconnected local variables disappear. Space assertion retains native assertion behavior rather than serializing active constraint frames. Discarding hidden constraints is equivalent to logical existential projection only under the relevant satisfiability hypothesis. No synthetic answer wrapper or full declaration-world table is introduced.

# 5 Proof obligations and their actual status

The generic Lean library `Mettapedia.Logic.ConstraintTypes` is the mathematical reference [6]. Its useful results include:

1. **Checker exactness.** `DeclarationTyping.check_iff` and `OpenTyping.compile_correct` relate executable abstract checkers to their declaration judgments.
2. **Staged exactness.** `CallTyping.frame_exact` and `PatternContracts.frame_exact` retain one assignment across input and result conditions.
3. **Compiled membership.** `DeterminedMembership.compiled_membership_correct` applies under its field-determination premises; whole-value nominal evidence is not constructor inversion.
4. **Vacuous-parameter elimination.** `UnsharedParameters.waive_joint_iff` requires coverage: the value has some proper type. The canonical ground-value model supplies this hypothesis to models containing its facts. This is not Reynolds parametricity.
5. **Local cost laws.** `CheckCost.incremental_exact` preserves verdicts after each new element; `entry_check_covers_suffixes` justifies reuse of a fixed element predicate on list suffixes.

These are conditional laws of abstract models. There is no completed refinement proof for the source normalizer, Prolog emitter, native attributes or the whole interpreter. In particular, the Lean checking-pattern extension is more general than the selected source policy; its alternative body-evaluation policy is not the retained Atom-template convention.

Finite trees are the mathematical value domain. Unrestricted native slots do not inspect cyclic host terms; checks that traverse structure test for cycles. Neither coverage nor list-cost theorems should be reported as theorems about arbitrary rational trees. Likewise, native watcher scheduling and sharing need tests in addition to the abstract binding laws.

# 6 Costs, compatibility and review

Checking all prefixes or suffixes of an $n$-element list performs $\sum_{k=1}^{n} k = n(n+1)/2$ element checks. For protected structural types, local suspensions check each newly bound part. For an eligible self-call on `cdr-atom` of an already checked list, the remaining list check can be reused; other inputs and result checks remain. These guarantees are narrow. General nominal or overloaded residual formulas can still rescan, and field extraction from a nominally typed value alone is not a proof that checks may be removed.

The measured comparison below uses identical typed programs, compiled MeTTa calls on both trees, and three interleaved repetitions against `stable-fixed 5d4b75e`. Loading and assertions are included for `nilbc`. Inference counts are less sensitive to host load than CPU time.

| Workload | Typed stable-fixed | Draft |
|---|---:|---:|
| 10,000 scalar calls | 90,005 | 60,005 |
| Counting, 200,000 steps | 2,000,014 | 1,000,009 |
| Constructor walk, 400 cells | 1,851,823 | 83,012 |
| Open-variable loop | 90,014 | 70,025 |
| `nilbc`, original shared signature | 20,100,081 | 19,486,151 |

The shared `$b` input/result constraint in `nilbc` is retained. Changing it to unrelated input and result parameters is a weaker contract, not a semantics-preserving optimization justified by a few passing examples. Returning the same term also need not preserve its nominal membership across declaration edits. No benchmark here removes that obligation to obtain a win.

Ten untyped execution controls match inference counts; this is not a claim of zero loading or mixed-dispatch overhead. Validation covers 112 runtime tests, 49 definition-order tests and 10 parser tests, in ordinary and optimized modes, plus 177 example files with 786 assertions and documented interactive exclusions.

Compatibility changes remain visible: indirect calls and later bindings are checked; overlapping type evidence does not duplicate effects; reflection-based derived classifications require explicit application guards. Mixed Atom declarations use the draft's unanimity policy. The compiler still preserves unanimous Atom-result templates. These choices require review on their own merits; generic type theory does not compel an upstream language change.

# References


[1] A. Mycroft and R. A. O'Keefe. *A polymorphic type system for Prolog.* Artificial Intelligence, 23(3), 1984. doi:10.1016/0004-3702(84)90017-1.

[2] J. Jaffar and J.-L. Lassez. *Constraint Logic Programming.* POPL, 1987. doi:10.1145/41625.41635.

[3] SWI-Prolog reference manual. Attributed variables.

[4] SWI-Prolog reference manual. `freeze/2`.

[5] SWI-Prolog reference manual. Impact of transactions.

[6] MeTTapedia. *ConstraintTypes* and *PeTTa.RuntimeTyping.* Source repository; the local development includes revisions `7e397154e6` and `63cddbdaa9`. Native conformance is not claimed.